

МИНОБРНАУКИ РОССИИ
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ БЮДЖЕТНОЕ ОБРАЗОВАТЕЛЬНОЕ
УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
“ВОРОНЕЖСКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ”
(ФГБОУ ВО «ВГУ»)

Факультет компьютерных наук

Кафедра программирования и информационных технологий

Разработка мобильного приложения для владельцев домашних питомцев
«PetDiary»

Курсовая работа

09.03.02 Информационные системы и технологии

Профиль «Программная инженерия в информационных системах»

Зав. кафедрой _____ д.ф.-м.н., доцент С.Д. Махортов

Обучающийся _____ ст. 3 курса Е.А. Аверкиева

Руководитель _____ асс. В.А. Ушаков

Воронеж 2026

СОДЕРЖАНИЕ

ОПРЕДЕЛЕНИЯ, ОБОЗНАЧЕНИЯ И СОКРАЩЕНИЯ	3
ВВЕДЕНИЕ	4
1 Предметная область и существующие решения.....	6
1.1 Описание предметной области и актуальности	6
1.2 Обзор аналогов	6
1.3.1 DogNote	7
1.3.2 Дневник питомца	8
1.3.3 АйЛапки! Дневник питомца	8
1.3.5 Результат сравнения анализа	9
2 Проектирование системы	11
2.1 Функциональные требования	11
2.2 Нефункциональные требования	13
2.3 Требование к системе	13
2.4 Архитектура разрабатываемой системы	15
2.4.1 Серверная часть приложения.....	15
2.4.2 База данных.....	15
2.4.3 Клиентская часть приложения	17
2.4.4 Взаимодействие клиент-сервер.....	18
3 Реализация разрабатываемой системы	20
3.1 Серверная часть приложения.....	20
3.2 Клиентская часть приложения.....	22
3.3.1 Экраны авторизации	22
3.3.3 Экран профиля питомца	25
3.3.4 Экран управления событиями.....	26
3.3.5 Экран карты	29
3.3.6 Панели навигации.....	30
ЗАКЛЮЧЕНИЕ	32
СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ	33

ОПРЕДЕЛЕНИЯ, ОБОЗНАЧЕНИЯ И СОКРАЩЕНИЯ

В настоящей работе применяют следующие термины с соответствующими определениями:

СУБД — система управления базами данных.

Фреймворк — (с англ. framework — «каркас, структура») обширная структура, предоставляющая набор инструментов, библиотек и правил для разработки приложений.

API (Application programming interface) — это программный интерфейс приложений, набор инструкций, который позволяет разным приложениям общаться между собой [1].

JSON (JavaScript Object Notation) — это текстовый формат обмена данными, который используется для хранения данных и их передачи между различными системами и приложениями [2].

ER-диаграмма — тип диаграммы, используемый для визуализации структуры базы данных, показывающий сущности и связи между ними.

ВВЕДЕНИЕ

В условиях современного мира наблюдается активная цифровизация всех сфер жизни человека — от покупок и общения до управления здоровьем и финансами. Мобильные технологии становятся неотъемлемой частью повседневности, предлагая удобные инструменты для решения самых разных задач. Не остаётся в стороне и сфера ухода за домашними питомцами, которая также нуждается в современных цифровых решениях.

По данным исследований аналитического центра ВЦИОМ от 26 ноября 2025 года 70% россиян имеют одного или нескольких домашних питомцев. А четверо из десяти россиян хотят завести питомца в будущем. Причем 48% от общего числа домашних животных занимают именно собаки [3].

Содержание питомца сопряжено с регулярными заботами: покупка корма, лакомств, систематические вакцинации, различные зооуслуги. Например, посещение ветеринара стало самой востребованной услугой, почти половина владельцев посещали ветклинику за последние полгода. Такая структура забот о питомце показывает, что уход за животным требует системного подхода к планированию. График прививок, даты обработки от паразитов, приём лекарств, плановые визиты к ветеринару — все эти события легко забыть в повседневной рутине. Следовательно, возникает необходимость в инструменте, который позволит владельцу не только фиксировать важные даты, но и получать своевременные напоминания.

Кроме того, исследование показывает, что в городской инфраструктуре для владельцев животных наиболее востребованы ветеринарные клиники и зоомагазины. Однако поиск ближайшей ветклиники или профильного магазина в незнакомом районе может быть затруднительным. Поэтому, помимо планирования событий, пользователю необходим инструмент для поиска зоо-услуг на карте.

Таким образом, разработка мобильного приложения для организации напоминаний о ключевых событиях в жизни питомца и быстрого поиска

профильных услуг является значимым и актуальным направлением в сфере мобильных технологий.

Целью данной курсовой работы является разработка мобильного приложения, предназначенного для владельцев домашних питомцев.

Задачами работы являются:

- анализ рынка мобильных приложений для владельцев домашних животных;

- 1) определение сильных и слабых сторон аналогов;

- 2) определение функций разрабатываемой системы;

- выбор средств разработки;

- проектирование архитектуры приложения и базы данных;

- реализация функций разрабатываемой системы.

1 Предметная область и существующие решения

1.1 Описание предметной области и актуальности

Предметной областью разработанной системы является цифровое сопровождение ухода за домашними животными с использованием мобильных технологий.

С развитием общества и изменением образа жизни роль домашних питомцев в семьях существенно трансформировалась. Питомцы все чаще воспринимаются не просто как животные, а как полноправные члены семьи, что закономерно повышает уровень ответственности владельцев за их здоровье, питание и общее благополучие.

Однако, значительная часть проблем со здоровьем у питомцев возникает из-за несвоевременного проведения профилактических мероприятий (вакцинации, обработки от паразитов и т.д.) или визитов к ветеринару, о которых владельцы попросту забывают в потоке повседневных дел. Одной из сложностей является фрагментация информации: данные о питомце хранятся в бумажных ветеринарных паспортах, напоминания ставятся в стандартном календаре телефона.

Таким образом, разработка мобильного приложения для комплексного учета и планирования событий, связанных с домашними питомцами, является значимым и актуальным направлением, отвечающим на реальные потребности современного владельца животного.

1.2 Обзор аналогов

Для выявления функциональных требований и конкурентных преимуществ проведем анализ нескольких популярных приложений для ведения дневника питомца.

Сравнение приложений будет проводиться по следующим критериям:

- создание профиля питомца;
- календарь событий с напоминаниями;
- работа в офлайн-режиме;
- интеграция с картографическим сервисом для поиска инфраструктуры для животных.

1.3.1 DogNote

Приложение DogNote - Pet journal предназначено для ведения журнала активности питомца и координации ухода за ним между членами семьи. Приложение помогает отслеживать события, связанные с питомцем, такие как тренировки, прогулки, визиты к ветеринару и другие важные моменты. Интерфейс приложения выполнен в минималистичном стиле с преобладанием светлых тонов и крупных иконок, что упрощает навигацию для пользователей всех возрастов.

Основные функции приложения включают:

- Создание семейного хаба для совместного доступа к информации о питомце;
- Лента активности питомца с отображением всех зарегистрированных событий;
- Напоминания и уведомления о вакцинации, приемах у ветеринара и других событиях;
- Добавление фотографий для сохранения памятных моментов;
- Отслеживание веса с визуализацией данных в виде графиков;
- Экспорт данных для сохранения и обмена информацией;

Можно выделить несколько недостатков приложения: в нем отсутствует работа без интернета (все данные синхронизируются с облачным сервером), в приложение не интегрирована картографическая служба — пользователь не может найти ближайшую ветклинику или зоомагазин прямо из интерфейса

приложения, а также после 14 дней пробного использования доступ к напоминаниям и семейному хабу ограничивается, остаётся только базовый просмотр ленты.

1.3.2 Дневник питомца

Мобильное приложение «Дневник питомца» ориентировано преимущественно на русскоязычную аудиторию и делает акцент на мониторинге здоровья животного. Приложение распространяется по модели freemium — базовые функции бесплатны, но расширенная медицинская аналитика доступна по подписке.

Основные функции приложения:

- трекер здоровья (специализированные поля для фиксации медицинских показателей);
- дневник активности (запись прогулок с фиксацией пройденного расстояния);
- напоминания;
- персонализация;
- графики и анализ данных;
- резервное копирование и экспорт данных;

1.3.3 АйЛапки! Дневник питомца

Приложение «АйЛапки» позиционируется как «первая социальная сеть и интеллектуальный дневник» для кошек, собак, хомяков и даже растений. Ключевое отличие этого аналога — смещение фокуса с утилитарных функций (напоминания, календарь) в сторону социального взаимодействия и обмена контентом. Приложение полностью бесплатно и монетизируется за счёт показа рекламы и донатов.

Основные функции данного приложения:

- профессиональный инструмент измерения частоты дыхания;
- напоминания;
- социальная лента (пользователи могут публиковать фотографии своих питомцев, ставить лайки, оставлять комментарии и подписываться друг на друг);
- онлайн-страница питомца;
- календарь для растений;
- облачное хранилище.

Мобильное приложение «АйЛапки!» подойдёт пользователям, которые хотят делиться фотографиями питомца и участвовать в сообществе, но оно не является полноценным инструментом планирования и поиска услуг.

1.3.5 Результат сравнения анализа

Результат, проведенного сравнения функциональных возможностей аналогов, представлен в таблице 1.

Таблица 1 - обзор аналогов

	Профиль питомца	Календарь событий	Офлайн-режим	Поиск клиник на карте
DogNote	+	+	—	—
Дневник питомца	+	+	+	—
АйЛапки!	+	+	+	—
PetDiary	+	+	+	+

Анализ таблицы 1 показывает, что разрабатываемое мобильное приложение занимает уникальную позицию на рынке, сочетая преимущества, которые по отдельности присутствуют у конкурентов, но редко объединены в одном решении.

Приложение не имеет прямых аналогов, которые сочетали бы

одновременно:

- создание детального профиля питомца с фотографией;
- ведение календаря событий;
- систему напоминаний о запланированных событиях;
- встроенный картографический модуль для оперативного поиска ветеринарной инфраструктуры;
- возможность полноценного использования без обязательной регистрации;

Таким образом, аналитическая часть работы подтвердила актуальность и практическую значимость разработки мобильного приложения «Pet Diary». Сформулированные выводы служат основанием для перехода к проектной части, в которой будут детально проработаны архитектура системы, структура базы данных и пользовательские сценарии.

2 Проектирование системы

2.1 Функциональные требования

Функциональные требования устанавливают перечень основных возможностей системы и определяют спектр действий, доступных пользователю при взаимодействии с программным продуктом.

Для определения основных функций разрабатываемого мобильного приложения были сформированы пользовательские сценарии с помощью диаграммы прецедентов.

Диаграмма прецедентов (англ. use-case diagram) диаграмма, описывающая, какой функционал разрабатываемой программной системы доступен каждой группе пользователей [4].

Данный метод позволяет наглядно представить взаимодействие пользователя с системой и выявить все функции. Спроектированная диаграмма прецедентов представлена на рисунке 1.

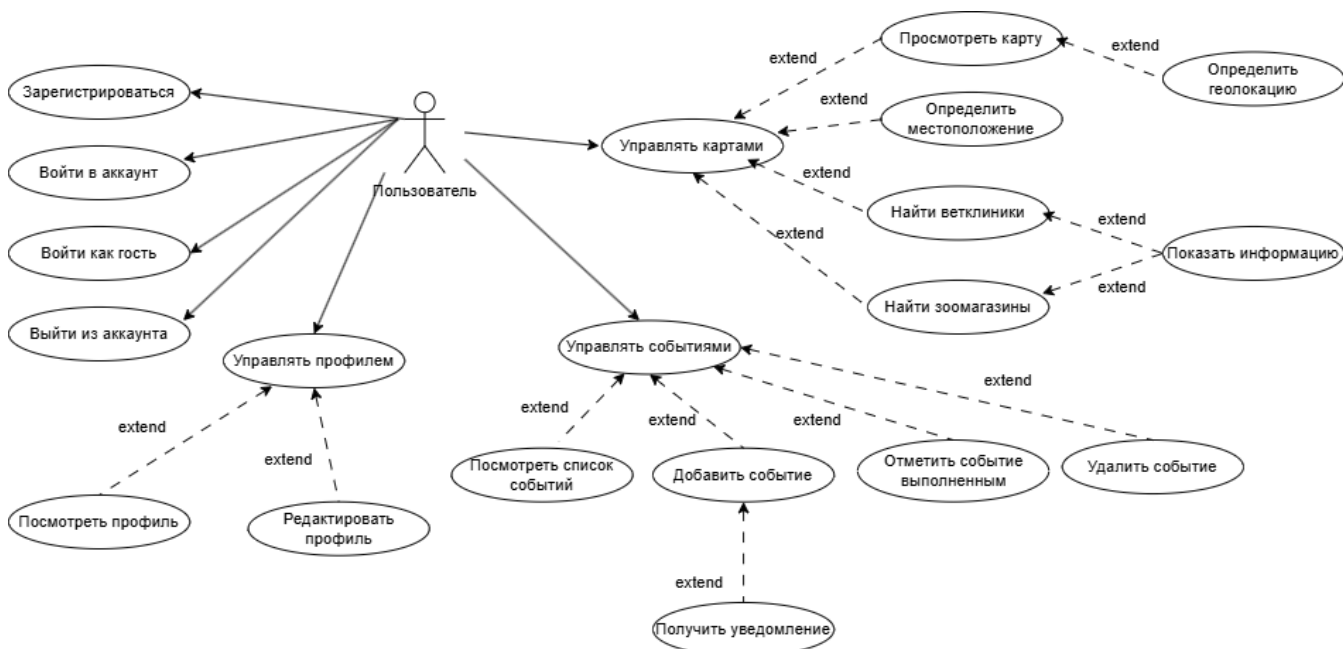


Рисунок 1 – Диаграмма прецедентов

В приложении реализован один вид пользователя – «Пользователь»,

который может работать как в авторизованном режиме, так и в гостевом. Для определения роли пользователя ему необходимо пройти процедуру аутентификации: зарегистрироваться, войти в существующий аккаунт или выбрать гостевой режим работы без регистрации.

На диаграмме выделены три основные группы функций: управление профилем, управление событиями и управление картами.

На основе спроектированной диаграммы прецедентов были детализированы следующие функциональные требования:

— Управление аутентификацией. Система должна предоставлять пользователю возможность регистрации с указанием email и пароля, входа в систему с использованием учётных данных, а также работы в гостевом режиме без обязательной регистрации. При регистрации система должна проверять уникальность email. После успешной аутентификации система должна генерировать JWT-токен для последующих запросов. Должна быть предусмотрена возможность выхода из аккаунта.

— Управление профилем питомца. Система должна позволять пользователю создавать и редактировать детальный профиль питомца. Система должна автоматически рассчитывать возраст питомца на основе даты рождения. Предусмотрена возможность загрузки и отображения фотографии питомца с локального устройства или из облачного хранилища.

— Управление событиями календаря. Система должна предоставлять работу с событиями: создание новых событий с указанием названия, описания, даты и времени; удаление событий; отметка событий как выполненных. События должны группироваться по датам. Система должна обеспечивать фильтрацию событий по статусу выполнения и дате.

— Система напоминаний. Система должна автоматически планировать локальные уведомления о предстоящих событиях за 15 минут до наступления события. Уведомления должны отображаться даже при закрытом приложении. При нажатии на уведомление должно происходить открытие приложения с переходом к соответствующему событию. Система

должна отменять уведомления для выполненных или удалённых событий.

— Интеграция с картографическим сервисом. Система должна предоставлять интерактивную карту с отображением текущего местоположения пользователя. Предусмотрена возможность поиска ветеринарных клиник и зоомагазинов поблизости с фильтрацией по категориям. При выборе места на карте система должна отображать информацию о месте: название, адрес, ссылку на место в приложении Яндекс.Карты.

2.2 Нефункциональные требования

Нефункциональные требования определяют критерии качества работы системы, ограничения и условия её эксплуатации. Для разрабатываемого мобильного приложения «PetDiary» были сформулированы следующие требования:

— все пароли пользователей должны храниться в базе данных в хешированном виде;

— валидация входных данных (Все данные, поступающие от клиента, должны проходить проверку на корректность перед обработкой);

— доступ к защищённым ресурсам API осуществляется только при наличии валидного JWT-токена, что предотвращает несанкционированный доступ к данным;

— адаптивный дизайн (интерфейс приложения должен автоматически подстраиваться под ширину экрана устройства).

2.3 Требование к системе

Для реализации серверной части приложения использованы следующие средства:

— язык программирования Java – строго типизированный объектно-

ориентированный язык программирования общего назначения [5];

— фреймворк Spring Boot – универсальный фреймворк с открытым исходным кодом для Java-платформы [6]. Он обеспечивает быструю разработку REST API, внедрение зависимостей и высокую производительность;

— СУБД PostgreSQL – это объектно-реляционная система управления базами данных, наиболее развитая из открытых СУБД в мире. Имеет открытый исходный код и является альтернативой коммерческим базам данных [7];

Для реализации клиентской части приложения будут использоваться следующие средства:

— Язык программирования Kotlin – кроссплатформенный, статически типизированный, объектно-ориентированный язык программирования, работающий поверх Java Virtual Machine [8];

— Yandex MapKit. Данное API предоставляет технологии и картографические данные для разработки мобильных приложений, использующих геолокацию;

— Яндекс.Диск REST API – для облачного хранения фотографий питомцев;

— компонент AlarmManager – системный сервис Android, применяемый для планирования локальных уведомлений о предстоящих событиях.

Таким образом, выбранные средства реализации позволяют создать масштабируемое решение, полностью отвечающее поставленным задачам и функциональным требованиям мобильного приложения «PetDiary».

2.4 Архитектура разрабатываемой системы

2.4.1 Серверная часть приложения

Серверная часть следует многоуровневой архитектуре:

— controller;

Слой контроллеров отвечает за прием REST-запросов от клиентского Android-приложения, передачу информации на обработку слою сервисов приложения и возврат соответствующего ответа пользователю в формате JSON. В разработанной системе реализованы три основных контроллера: AuthController (обрабатывает запросы аутентификации и регистрации по пути /api/auth), ProfileController (управляет запросами профиля питомца по пути /api/profile), EventController (обрабатывает запросы событий календаря по пути /api/events).

— service;

Сервисы представляют собой всю бизнес-логику приложения. Здесь происходит обработка данных, валидация запросов, использование уровня репозитория для доступа к базе данных, а также генерация JWT-токенов и работа с системой уведомлений. В системе реализованы следующие сервисы: AuthService, EventService, JwtService (генерирует и валидирует JSON Web Tokens для аутентификации).

— repository.

Слой JPA-репозитория позволяет на уровне абстракции работать с базой данных PostgreSQL. С помощью этого слоя происходит работа с сущностями базы данных: User, PetProfile и Event. Репозитории предоставляют методы для выполнения CRUD-операций и сложных запросов.

2.4.2 База данных

Проектирование базы данных выполнялось с соблюдением принципов нормализации. База данных находится в третьей нормальной форме, что

позволяет избежать избыточности данных и аномалий при их обновлении.
Спроектированная база данных представлена на рисунке 2.

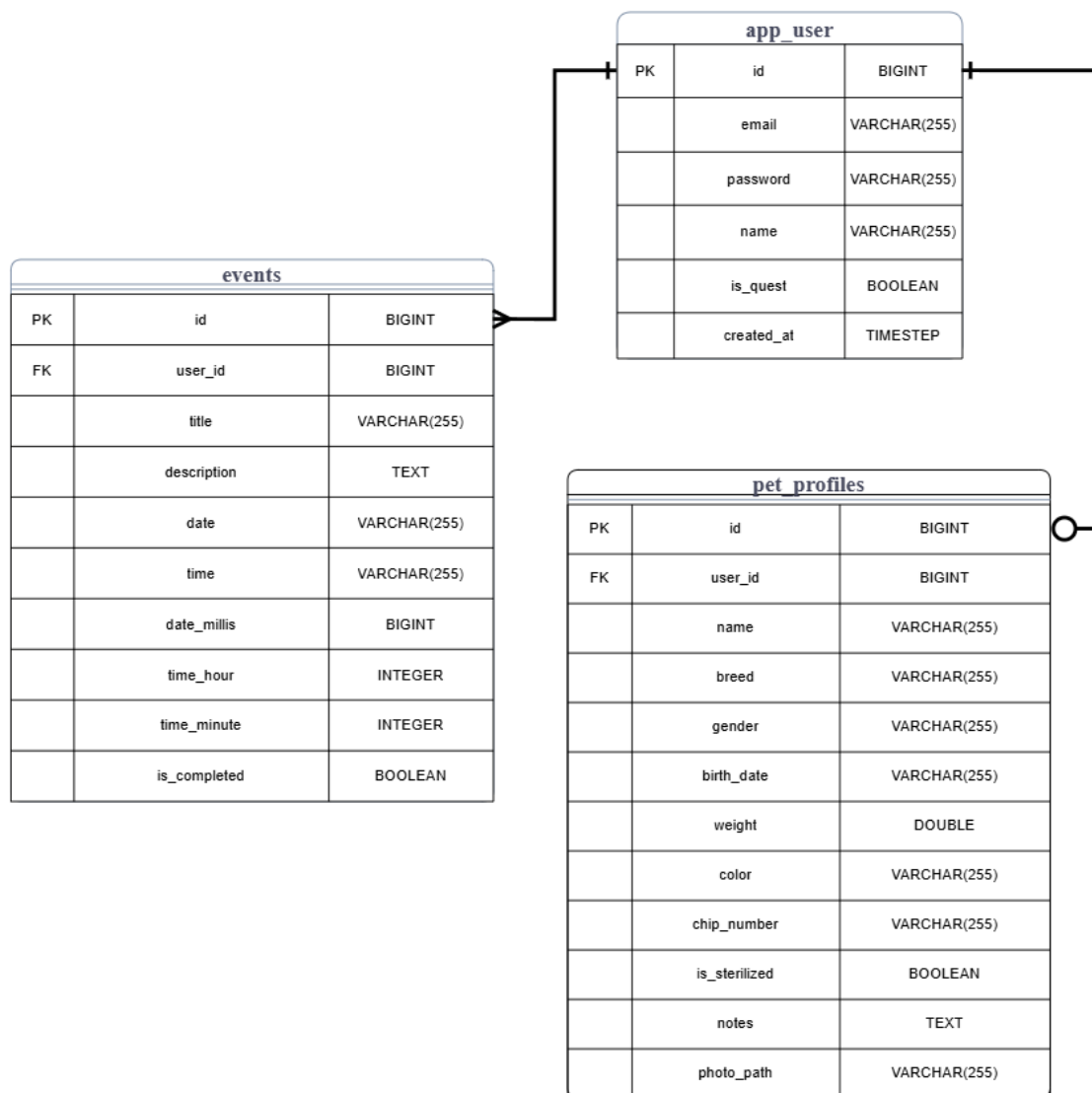


Рисунок 2 - ER диаграмма

Спроектированная база данных содержит три основные сущности:

— `app_user` — хранит данные о пользователях приложения (идентификатор, email, хеш пароля, имя, флаг гостевого режима, дата создания аккаунта);

— `pet_profiles` — содержит информацию о питомце (идентификатор, кличка, порода, пол, дата рождения, вес, окрас, номер чипа, флаг

стерилизации, заметки, путь к фотографии);

— events – хранит события календаря (идентификатор, название, описание, дата, время, флаг выполнения).

Между сущностями app_user и pet_profiles установлена связь «один-к-одному» (1:1). Каждый пользователь может иметь только один профиль питомца, и каждый профиль питомца принадлежит только одному пользователю. Эта связь реализована через внешний ключ user_id в таблице pet_profiles.

Между сущностями app_user и events установлена связь «один-ко-многим» (1:M). Один пользователь может иметь множество событий в календаре или не иметь их вовсе, но каждое событие обязательно принадлежит только одному пользователю. Эта связь также реализована через внешний ключ user_id в таблице events, который ссылается на первичный ключ таблицы app_user.

2.4.3 Клиентская часть приложения

Клиентская часть приложения организована в соответствии с архитектурным паттерном MVVM (Model-View-ViewModel). Данный паттерн обеспечивает чёткое разделение ответственности и упрощает тестирование кода.

Модели представляют собой классы, которые описывают структуру данных, получаемых с сервера или сохраняемых локально. Примеры моделей: Event, PetProfile. Модели не содержат бизнес-логики — их единственная задача заключается в хранении данных.

View. В данном приложении ими выступают Activity и Fragment. Они отвечают только за отображение пользовательского интерфейса и передачу действий пользователя во ViewModel. View не принимает решений и не содержит логики.

ViewModel – связующее звено между View и Model. Оно хранит данные

экрана, обрабатывает действия пользователя (нажатия кнопок, ввод текста) и передаёт результаты обратно на экран через LiveData.

Репозитории – посредники между ViewModel и источниками данных. Они инкапсулируют логику выбора источника: например, если пользователь находится в гостевом режиме, репозиторий берёт данные из SharedPreferences, а если пользователь авторизован — отправляет сетевой запрос. Это позволяет ViewModel не знать о деталях получения данных и легко подменять источники при необходимости.

Таким образом, применение MVVM-архитектуры позволяет создать приложение с чёткой структурой кода, которое легко расширять новыми функциями и поддерживать в рабочем состоянии.

2.4.4 Взаимодействие клиент-сервер

Взаимодействие между клиентом и сервером осуществляется по протоколу HTTP. Все запросы и ответы имеют формат JSON. Клиентская часть использует библиотеку Retrofit для описания API-интерфейсов и автоматической сериализации объектов в JSON и обратно.

Процесс аутентификации построен на основе JWT-токенов и включает в себя следующие этапы:

1. Пользователь отправляет на сервер свои учётные данные (email и пароль) через адрес API /api/auth/login.
2. Сервер проверяет корректность данных через слой репозитория, генерирует JWT-токен и возвращает его клиенту вместе с информацией о пользователе (userId, email, флаг гостя).
3. Клиент получает ответ и переходит на главный экран приложения.

Процессы обмена данными и взаимодействия между компонентами системы удобно и наглядно отображать с помощью диаграммы последовательности.

Диаграмма последовательности (sequence diagram) — это наглядное

представление совокупности различных элементов модели системы, показывающее, как и в каком порядке они взаимодействуют между собой во времени [9].

Данный процесс аутентификации показан на диаграмме последовательностей на рисунке 3.

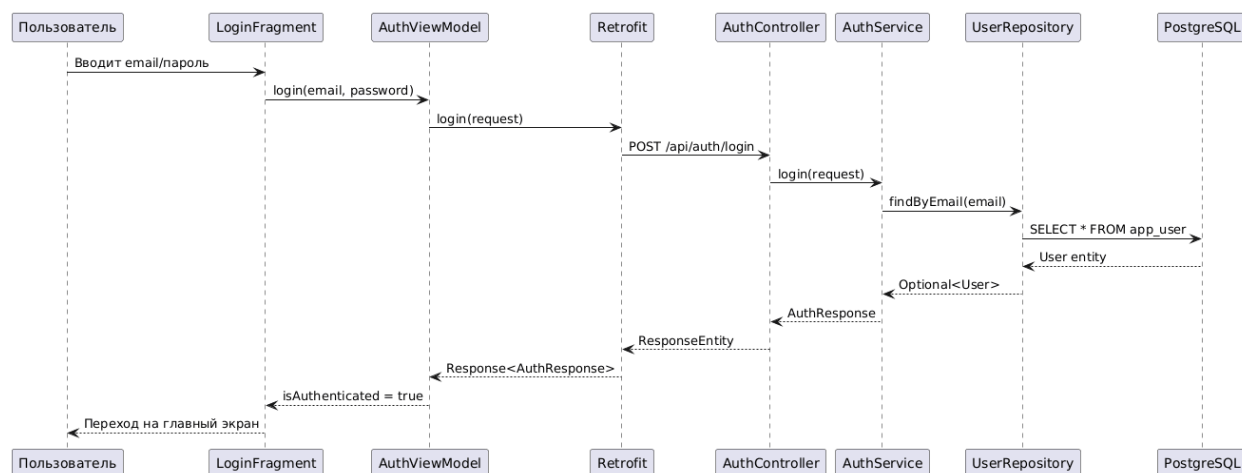


Рисунок 3 – Диаграмма последовательностей (Авторизация)

После получения JWT-токена клиентское приложение сохраняет его и использует для всех последующих запросов к серверу. Токен добавляется в заголовок каждого запроса, что позволяет серверу идентифицировать пользователя без повторной передачи логина и пароля. Срок действия токена ограничен, что повышает безопасность приложения.

Гостевой режим работает без токена, что позволяет использовать основные функции приложения без регистрации.

3 Реализация разрабатываемой системы

3.1 Серверная часть приложения

В разработанном приложении реализована архитектура аутентификации на основе технологии JWT (JSON Web Token). Веб-токен JSON — это открытый стандарт, который определяет компактный и автономный способ безопасной передачи информации между сторонами в виде объекта JSON [10].

При успешной регистрации или входе сервер генерирует уникальный токен и возвращает его клиенту. Во всех последующих запросах к защищенным ресурсам API клиент обязан передавать этот токен в заголовке Authorization. Серверный фильтр JwtAuthFilter проверяет цифровую подпись токена и извлекает из него данные пользователя. Срок жизни токена ограничен.

В серверной части приложения была реализована OpenAPI — спецификация для описания и документирования API [11]. Она автоматически генерирует интерактивную документацию API на основе аннотаций в коде контроллеров. Это позволяет наглядно отображать структуру всех доступных адресов запросов, форматы JSON-запросов и ответов, а также тестировать работу сервера напрямую из браузера, без необходимости запуска мобильного приложения.

Было выделено три основных функциональных модуля.

Модуль «Авторизация» реализован классом AuthController, который обрабатывает запросы по пути /api/auth. Данный модуль предоставляет методы API для регистрации нового пользователя (/register), входа в систему (/login) и создания гостевой сессии (/guest), что позволяет начать использование приложения без обязательной регистрации. Открытая спецификация данного модуля представлена на Рисунке 4.

auth-controller		^
POST	/api/auth/register	▼
POST	/api/auth/login	▼
POST	/api/auth/guest	▼

Рисунок 4 - OpenAPI модуля «Авторизация»

Модуль «Профиль питомца» отвечает за создание и редактирование детальной карточки питомца. Класс ProfileController обрабатывает запросы по пути /api/profile. Сервис ProfileService реализует логику сохранения данных (включая привязку пути к фотографии), а PetProfileRepository обеспечивает сохранение сущности в базе данных с установлением связи «один-к-одному» с пользователем. Спецификация модуля представлена на Рисунке 5.

profile-controller		^
GET	/api/profile	▼
POST	/api/profile	▼
GET	/api/profile/exists	▼

Рисунок 5 - OpenAPI модуля «Профиль питомца»

Модуль «События» управляет календарем событий питомца. Класс EventController предоставляет полный цикл CRUD-операций по пути /api/events, а также специализированные методы для получения активных и сегодняшних событий. EventService содержит логику фильтрации событий по времени. Спецификация модуля представлена на Рисунке 6.

event-controller		^
PUT	/api/events/{id}	▼
DELETE	/api/events/{id}	▼
GET	/api/events	▼
POST	/api/events	▼
PATCH	/api/events/{id}/toggle	▼
GET	/api/events/today	▼
GET	/api/events/active	▼
DELETE	/api/events/past	▼

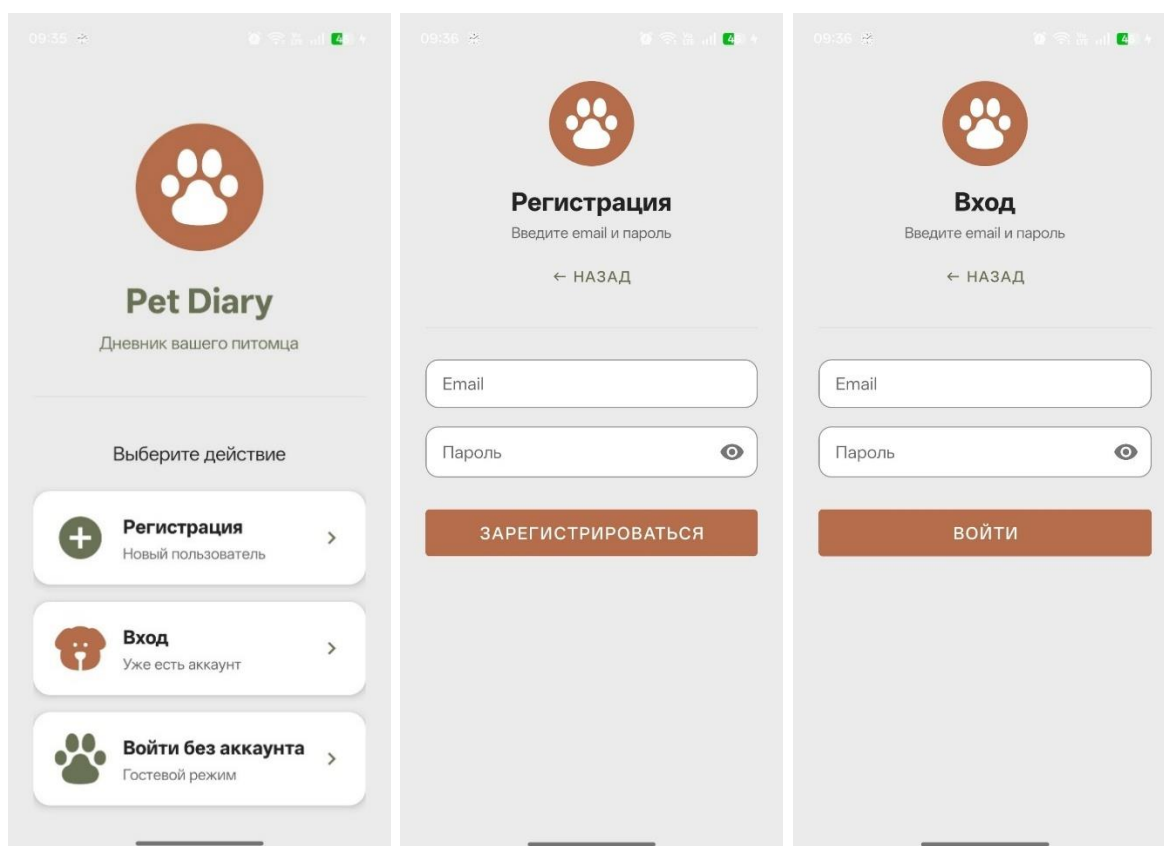
Рисунок 6 - OpenAPI модуля «События»

Таким образом, серверная часть обеспечивает надежное, безопасное и структурированное хранение данных.

3.2 Клиентская часть приложения

3.3.1 Экраны авторизации

При первом запуске пользователь попадает на экран выбора (AuthChoiceFragment), где может зарегистрироваться, войти или использовать приложение в гостевом режиме. Демонстрация внешнего вида экрана представлена на рисунке 7.



а

б

в

а – экран выбора авторизации, б – экран авторизации, в – экран входа

Рисунок 7 - Экраны авторизации

3.3.2 Главный экран

Главный экран (HomeFragment) является стартовым после успешной аутентификации. Он состоит из трёх основных блоков.

Блок «Мой питомец» представляет собой карточку с фотографией питомца, его именем, породой и возрастом. Возраст рассчитывается динамически на основе даты рождения. Если дата рождения не указана, блок возраста не отображается. Фотография загружается либо из локального файла (если пользователь гость), либо по ссылке из облачного хранилища.

Блок «События на сегодня» отображает события, запланированные на текущую дату. Если событий нет, выводится сообщение «На сегодня событий нет». Если событий больше пяти, в конце списка добавляется текст с указанием количества оставшихся событий.

Блок «Совет дня» содержит случайный совет по уходу за домашними питомцами. Советы хранятся в ресурсном файле в виде трёх массивов (эмодзи, заголовки, тексты). При каждом открытии экрана выбирается случайный индекс, и на экран выводятся соответствующие значения. Такой подход позволяет легко пополнять коллекцию советов без изменения кода.

HomeViewModel загружает данные при создании и обновляет их в методе `onResume()` фрагмента. Для этого используются два метода: `loadPetProfile()` (загружает профиль питомца из сети или локального хранилища) и `loadTodayEvents()` (загружает события на сегодня).

Демонстрация внешнего вида экрана представлена на рисунке 8.

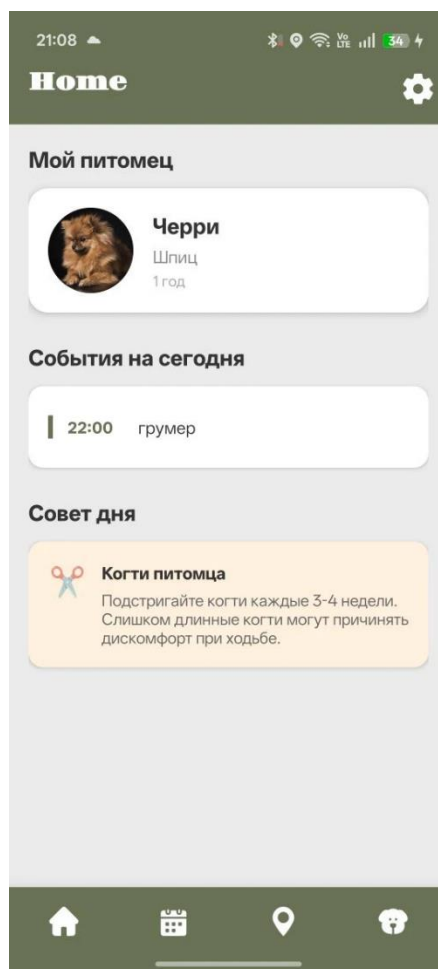


Рисунок 8 – Главный экран

3.3.3 Экран профиля питомца

Экрана профиля (ProfileFragment) позволяет пользователю создать детальную карточку питомца, загрузить его фотографию и в случае необходимости отредактировать введенные данные. Экран поддерживает два режима отображения: режим редактирования (при первом запуске или нажатии кнопки изменения) и режим чтения (после сохранения профиля).

Имеются следующие элементы экрана:

- Карточка с круглым изображением (для отображения фотографии питомца);
- Текстовое поле для ввода клички питомца;
- Выпадающий список с для выбора породы;
- Группа переключателей для выбора пола питомца («Мальчик» / «Девочка»);
- Поле для выбора даты рождения;
- Текстовое поле для ввода веса питомца;
- Текстовое поле для ввода окраса;
- Текстовое поле для ввода номера чипа;
- Переключатель для отметки статуса стерилизации;
- Текстовое поле для ввода дополнительных заметок;
- Кнопка «Сохранить» (в режиме редактирования);
- Кнопка «Редактировать» (в режиме чтения).

При нажатии на кнопку «Сохранить» происходит проверка корректности введенных данных. В случае если поле клички питомца пустое, выводится уведомление об ошибке, так как это обязательное поле для ввода при создании профиля питомца. При успешной проверке данные отправляются на сервер (для авторизованных пользователей) или сохраняются локально (в гостевом режиме), после чего экран переключается в режим чтения, где данные отображаются в виде текстовых полей без возможности редактирования. Повторное нажатие на кнопку «Изменить» возвращает экран

в режим ввода.

Демонстрация внешнего вида экрана представлена на Рисунке 9.

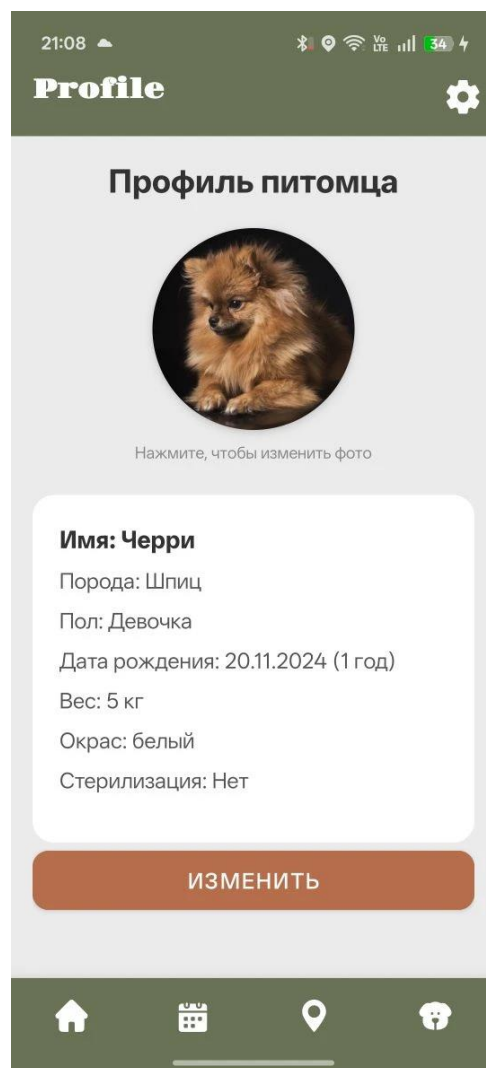


Рисунок 9 – Экран профиля питомца

3.3.4 Экран управления событиями

Данный экран (CalendarFragment) реализует основную функциональность приложения — отображение и управление событиями.

Он отображает события, сгруппированные по датам («Сегодня», «Завтра», конкретная дата) с помощью кастомного GroupedEventAdapter. Добавление события осуществляется при нажатии на кнопку «+ Добавить» через диалоговое окно с выбором даты и времени. Демонстрация диалогового

окна для создания события представлена на Рисунке 10.

20:29

Новое событие

Что нужно сделать?

Описание (необязательно)

0/200

ВЫБРАТЬ ВРЕМЯ

Уведомление придёт за 15 минут

Выберите дату:

Июнь 2026 г. >

П	В	С	Ч	П	С	В
1	2	3	4	5	6	7
8	9	10	11	12	13	14
15	16	17	18	19	20	21
22	23	24	25	26	27	28
29	30					

ОТМЕНА СОХРАНИТЬ

Рисунок 10 – Диалоговое окно создания события

Демонстрация внешнего вида экрана управления событиями, представлена на Рисунке 11.

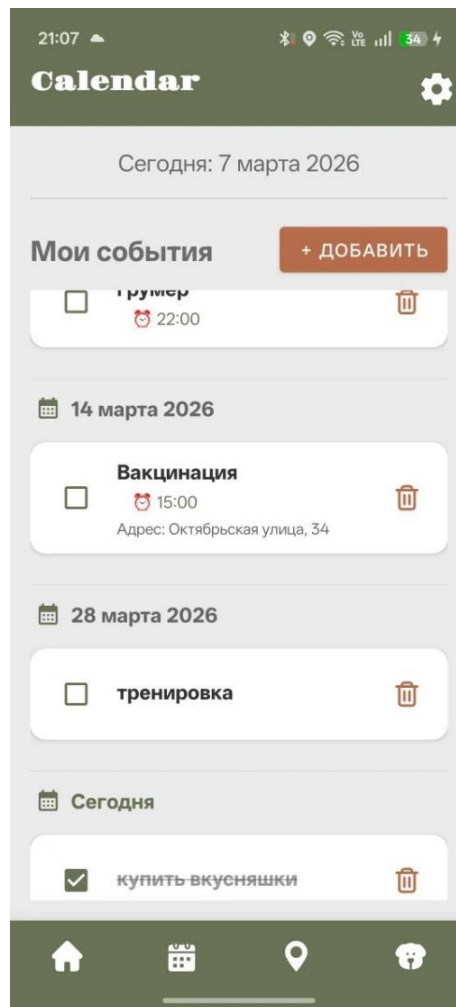


Рисунок 11 – Экран управления событиями

При сохранении события вызывается NotificationService, который вычисляет время срабатывания (за 15 минут до события) и планирует его через AlarmManager.setAlarmClock(), который гарантирует точное срабатывание даже в режиме энергосбережения.

Внешний вид системного уведомления, отображаемого за 15 минут до запланированного события, представлен на рисунке 12.

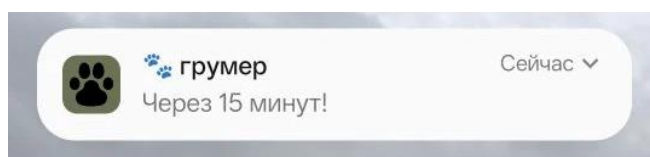
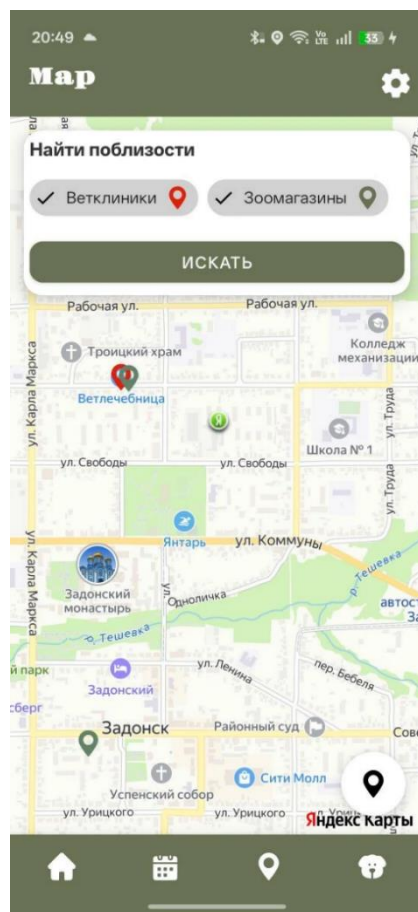


Рисунок 12 – Системное уведомление

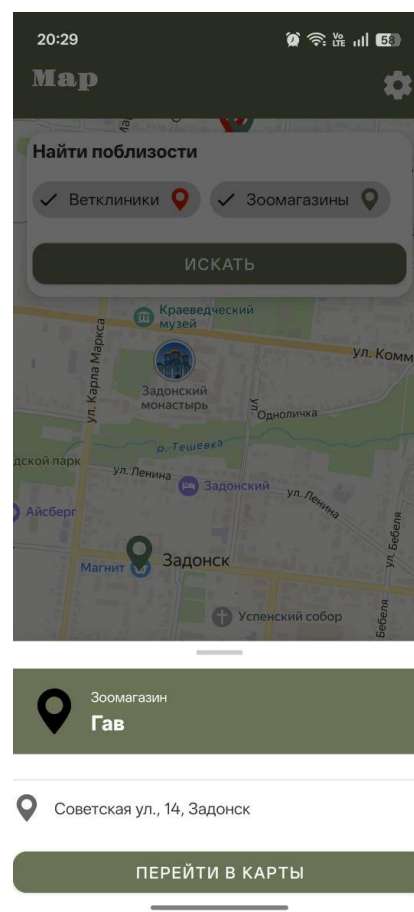
3.3.5 Экран карты

Экран карты (MapFragment) интегрирован с Яндекс.Картами (MapKit). При запуске экрана проверяется наличие разрешения на геолокацию пользователя (ACCESS_FINE_LOCATION). Если разрешение не предоставлено, оно запрашивается у пользователя. После получения разрешения создаётся UserLocationLayer, который отображает текущее местоположение пользователя на карте.

Пользователь может выбрать категории поиска инфраструктуры: «Ветклиники» и «Зоомагазины». При нажатии кнопки «Искать» формируется запрос к SearchManager Яндекс.Карт. В результате, на карту добавляются метки объектов на видимой области карты. Иконка метки зависит от типа объекта: красный пин для ветклиник, зелёный — для зоомагазинов. При нажатии на метку открывается окно с названием, адресом и кнопкой «Перейти в карты», необходимой для открытия данного объекта через приложение Яндекс.Карт. Демонстрация экрана карты с результатами поиска, а также окно с отображением информации об объекте представлены на рисунке 12.



а



б

а – экран карты, б – окно отображения информации

Рисунок 11 - Экраны карты

3.3.6 Панели навигации

На всех основных экранах приложения присутствуют верхняя и нижняя панели навигации.

В левой части верхней панели отображается название текущего экрана (в зависимости от экрана высвечивается: Home, Calendar, Map, Profile), в правой – расположена кнопка настроек. При нажатии на иконку открывается диалоговое окно с пунктом «Выйти из аккаунта».

Нижняя панель навигации расположена соответственно в нижней части экрана и обеспечивает быстрый переход между четырьмя основными

разделами приложения: главным экраном (HomeFragment), экраном управления событиями (CalendarFragment), экраном карты (MapFragment) и экраном профиля питомца (ProfileFragment). Навигация реализована с использованием компонента Navigation Component, что обеспечивает правильную обработку нажатий и корректное управление стеком фрагментов. При нажатии на иконку происходит переход к соответствующему фрагменту, а активный пункт меню визуально выделяется.

ЗАКЛЮЧЕНИЕ

В результате выполненной курсовой работы было разработано мобильное приложение «PetDiary» для владельцев домашних животных.

В рамках работы был проведён анализ рынка мобильных приложений для владельцев домашних животных, в ходе которого рассмотрены существующие аналоги, определены их сильные и слабые стороны, а также сформулированы функции разрабатываемой системы.

Был осуществлён выбор средств разработки: для серверной части выбраны Java 17, Spring Boot, PostgreSQL и JWT-аутентификация; для клиентской части — Kotlin, Android SDK, архитектурный паттерн MVVM, а также внешние сервисы Яндекс.Карты и Яндекс.Диск.

Выполнено проектирование архитектуры приложения и базы данных: серверная часть построена по многоуровневому принципу (контроллеры, сервисы, репозитории), база данных приведена к третьей нормальной форме, клиентская часть организована согласно паттерну MVVM.

Осуществлена реализация функций разрабатываемой системы: разработан пользовательский интерфейс для интуитивного взаимодействия с приложением, реализованы модули аутентификации (включая гостевой режим), управления событиями с напоминаниями, карты с поиском ветеринарных клиник и зоомагазинов, а также профиля питомца с возможностью загрузки фотографий.

Таким образом, цель работы достигнута, поставленные задачи решены в полном объёме.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Что такое API и что о нём нужно знать веб-разработчику. – Текст. : электронный // Яндекс Практикум: [сайт]. – 2022. – URL: <https://practicum.yandex.ru/blog/chto-takoe-api/a> (дата обращения 03.06.2026).
2. Что такое формат JSON. – Текст. : электронный // Яндекс Практикум: [сайт]. – 2025. – URL: <https://practicum.yandex.ru/blog/chto-takoe-format-json/> (дата обращения 03.06.2026).
3. Кошки, собаки и все-все-все!. – Текст. : электронный // ВЦИОМ. Новости: [сайт]. – 2025. – URL: <https://wciom.ru/analytical-reviews/analiticheskii-obzor/koshki-sobaki-i-vse-vse-vse> (дата обращения 01.06.2026).
4. Использование диаграммы вариантов использования UML при проектировании программного обеспечения // Хабр : [сайт]. – 2021. – URL: <https://habr.com/ru/articles/566218/> (03.06.2026).
5. Java - википедия. – Текст. : электронный // Википедия : [сайт]. – 2025. – URL: <https://ru.wikipedia.org/wiki/Java> (дата обращения 01.06.2026).
6. Что такое Spring Framework? От внедрения зависимостей до Web MVC. – Текст. : электронный // Хабр: [сайт]. – 2020. – URL: <https://habr.com/ru/articles/490586/> (дата обращения 01.06.2026).
7. PostgreSQL. – Текст. : электронный // SkillFactory Media: [сайт]. – 2024. – URL: <https://blog.skillfactory.ru/glossary/postgresql/> (дата обращения 01.06.2026).
8. Kotlin - википедия. – Текст. : электронный // Википедия : [сайт]. – 2026. – URL: <https://ru.wikipedia.org/wiki/Kotlin> (дата обращения 01.06.2026).
9. Диаграммы последовательности — простой способ управления процессами для аналитиков. – Текст. : электронный // Яндекс Практикум: [сайт]. – 2024. – URL: <https://practicum.yandex.ru/blog/sequence-diagram/> (дата обращения 05.06.2026).

10. Introduction to JSON WebToken. – Текст : электронный // JWT Debugger : [сайт]. – 2025. – URL: <https://jwt.io/introduction> (дата обращения: 03.06.2026).

11. Введение в OpenAPI: ёмко и полезно о важном: [сайт]. – 2025. – URL: <https://habr.com/ru/companies/docdoc/articles/880476/> (дата обращения: 01.06.2026).